

Parallel Text Search

Zwick, Patrick

patrick.zwick@htwg-konstanz.de

Döbele, Max

max.doebele@htwg-konstanz.de

06 February 2026

Abstract

1 Introduction

2 Testing Data

For testing, we use a collection of up to 98 books from Project Gutenberg, which we concatenate into a single text containing over 82 million characters. We perform queries using, at most, the 100 most common English words. Both the number of books and the number of queries can be varied to evaluate performance under different conditions.

3 Sequential Implementations

Goals of the sequential implementations: Simple, easy to parallize, as few dependencies as possible

3.1 Reference Implementation

To verify correctness, we use a simple baseline implementation, denoted `std`, which relies on `std::string::find()` for substring matching (see ??).

```
std::vector<std::vector<size_t>>>
find_std(const std::string &text, const std::vector<std::string> &queries) {
    std::vector<std::vector<size_t>>> indices(queries.size());
    for (size_t i = 0; i < queries.size(); ++i) {
        const std::string &query = queries[i];
        auto pos = text.find(query);
        while (pos != std::string::npos) {
            indices[i].push_back(pos);
            pos = text.find(query, pos + 1);
        }
    }
    return indices;
}
```

Listing 1: `find_std()` in `std_text_search.cpp`

3.2 Candidate Selection and Validation

The first implementation is divided into two distinct phases:

1. Identification of candidate positions for potential occurrences based on a simple filtering criterion. A position is considered a candidate if three characters match: the first character of the query, the middle character, and the last character.
2. Verification of each candidate position to determine whether it constitutes a valid occurrence.

The initial approach, denoted `candidate_v1`, performs candidate identification followed by validation separately for each query (see ??). The vector is cleared after each query and reused for the next, avoiding unnecessary allocations.

```

void find_candidates(const size_t text_length, std::vector<size_t> &results,
                    const std::string &text, const std::string &query) {
    const auto query_length = query.length();
    const auto mid = query_length >> 1;
    const auto end = query_length - 1;
    for (size_t i = 0; i < text_length - query_length; ++i) {
        if (text[i] == query[0] && text[i + mid] == query[mid] &&
            text[i + end] == query[end]) {
            results.push_back(i);
        }
    }
}

bool test_candidate(const size_t index, const std::string &text,
                   const std::string &query) {
    return std::memcmp(text.data() + index, query.data(), query.size()) == 0;
}

std::vector<std::vector<size_t>>
find_candidate_v1(const std::string &text,
                 const std::vector<std::string> &queries) {
    std::vector<std::vector<size_t>> indices(queries.size());
    const auto text_length = text.length();
    std::vector<size_t> results;
    for (size_t i = 0; i < queries.size(); ++i) {
        const std::string &query = queries[i];
        find_candidates(text_length, results, text, query);
        for (const auto &result : results) {
            if (test_candidate(result, text, query)) {
                indices[i].push_back(result);
            }
        }
        std::memset(results.data(), 0, results.size() * sizeof(size_t));
    }

    return indices;
}

```

Listing 2: `find_candidates()`, `test_candidate()` and `find_candidate_v1()` in `candidate_v1_text_search.cpp`

3.2.1 Using an Array Instead of `std::vector`

Since a `std::vector` may need to repeatedly grow its underlying storage and copy existing elements when its capacity is exceeded, this can introduce a significant performance bottleneck, at least for the first query. To avoid this overhead, the `candidate_v2` implementation instead pre-allocates an array with a size equal to the length of the text. In the worst case, every position in the text could be a valid candidate, so this allocation is sufficient to store all candidates.

3.2.2 Using a Bitmask

To improve CPU cache utilization and reduce the memory footprint, the `candidate_v3` implementation represents the candidate buffer as a *bitmask*, where each bit set to 1 denotes a valid candidate position. The bitmask is stored as an array of unsigned 64-bit integers, allowing 64 candidates to be packed into a single word.

Let m denote the length of the text. To represent candidate positions for a query of any length, we require

$$h = \left\lceil \frac{m + 63}{64} \right\rceil$$

64-bit words in the bitmask.

For a candidate at position k , the word index w of the corresponding 64-bit word B_w in the bitmask is obtained by dividing k by 64 which can be efficiently computed by a right shift of 6 bits:

$$w = \frac{k}{64} = k \gg 6.$$

The bit position b within the word B_w is given by taking k modulo 64, which can be efficiently computed by a bitwise AND with 63:

$$b = k \bmod 64 = k \& 63.$$

To set this bit, we construct a temporary value with only the target bit active by shifting the integer constant 1 to the left by the bit position b , and then update the mask word using the bitwise OR:

$$B_w \leftarrow B_w \mid (1 \ll b).$$

To reconstruct candidate positions from the bitmask, we iterate over all 64-bit words and extract the indices of the set bits.

For a given word index w , let B_w again denote the corresponding 64-bit word. If $B_w = 0$, the word contains no valid candidates. Otherwise, we repeatedly extract the least significant set bit until B_w becomes zero.

The position of this bit within the word is obtained using the number of trailing zeros:

$$b = \text{ctz}(B_w),$$

where ctz denotes the count of trailing zero bits.

The corresponding global candidate index is then

$$k = 64w + b.$$

After processing a candidate, the lowest set bit is cleared using the operation

$$B_w \leftarrow B_w \& (B_w - 1),$$

which removes the extracted bit and guarantees termination after all set bits have been processed.

This approach ensures that candidate extraction runs in time proportional to the number of valid candidates rather than the size of the bitmask.

3.2.3 Using a Bitmask for All Queries Together

Instead of allocating a separate bitmask for each query and clearing it after processing, the `candidate_v4` implementation uses single bitmask large enough to store candidate positions for *all queries* simultaneously.

For n queries, the total bitmask size becomes

$$h_{\text{total}} = n \cdot h,$$

where h is the number of 64-bit words needed for a single query.

To access the word corresponding to a candidate at position k for query q , we first compute the offset for that query:

$$l = q \cdot h.$$

The global word index in the combined bitmask is then

$$w_{\text{total}} = l + (k \gg 6),$$

where, as before, $k \gg 6$ gives the word index within the query's portion of the bitmask.

Reconstructing candidate positions proceeds in a similar manner. We iterate over all queries q , and within each query, over all words w from 0 to $h - 1$. For each word index, the offset-adjusted index is

$$w_{\text{total}} = l + w.$$

Candidate positions are then extracted from the set bits of $B_{w_{\text{total}}}$ using the same approach described previously.

3.3 Rolling Hash Approach Rabin-Karp Algorithm

Assume we are searching for the word "hello" in a large text with millions of characters. Using a naive approach, we would have to check up to all 5 characters for every single position in the text. This results in a time complexity of:

$$O(N \cdot M)$$

where:

- N = Length of the text
- M = Length of the searched word (query)

To avoid comparing every single character for every shift, we convert the search terms into a numerical value and compare them directly. Time complexity of calculating the Hash is:

$$O(M)$$

This results in an **average** time complexity of:

$$O(N + M)$$

It is possible that two different strings result in the same hash-value therefore, we have to consider for **worst-case** that every hit leads to a hash collision. For that case the time complexity is:

$$O(N \cdot M)$$

Calculation of the query **hash value**:

$$\text{query_hash} = q_0 \cdot B^{M-1} + q_1 \cdot B^{M-2} + \dots + q_{M-1} \cdot B^0$$

where B is a prime number (PRIME) acting as the base, and q_i is the numerical value of the character.

For a window of length M starting at position j in the text, the hash value is:

$$\text{window_hash}_j = t_j \cdot B^{M-1} + t_{j+1} \cdot B^{M-2} + \dots + t_{j+M-1} \cdot B^0.$$

To ensure we do not have to recalculate the hash value from scratch for every position in the text, we use the **Rabin-Karp Rolling Hash** algorithm[?]:

The previous window hash is:

$$\text{window_hash}_{j-1} = t_{j-1} \cdot B^{M-1} + t_j \cdot B^{M-2} + \dots + t_{j+M-2}.$$

1. **Subtract** the most significant character:

$$\text{window_hash}'_{j-1} = \text{window_hash}_{j-1} - t_{j-1} \cdot B^{M-1}.$$

2. **Shift** the window by multiplying with the PRIME:

$$\text{window_hash}''_{j-1} = \text{window_hash}'_{j-1} \cdot B.$$

3. **Add** the new character:

$$\text{window_hash}_j = \text{window_hash}''_{j-1} + t_{j+M-1}.$$

To make sure the actual text is equal to the query, we double-check with `text.compare()` (see ??). This Problem where two strings return the same hash-value is caused by the $\text{mod } 2^{64}$ nature of `uint64_t` (no negative values).

```
if (window_hash == query_hash) {
    if (text.compare(j, m, query) == 0) {
        indices[i].push_back(j);
    }
}
```

Listing 3: Collision verification using `compare()`

3.3.1 Grouping queries by length for enhanced performance

The first implementation `hash_v1.text_search.cpp` needs to scan the entire text separately for each query. Given a large text (e.g., approx. 84 MB) that exceeds the CPU's L1 and L2 cache capacities, this leads to a bottleneck as the text must be reloaded from RAM for every single query. To optimize memory bandwidth and improve temporal cache locality, `hash_v2` groups all queries by their length (see ??). The Rabin-Karp rolling hash is dependent on a fixed window size, this grouping allows the algorithm to scan the text only once for all queries of the same length.

```
std::unordered_map<size_t, std::vector<size_t>> length_groups;
for (size_t i = 0; i < queries.size(); ++i) {
    length_groups[queries[i].length()].push_back(i);
}
```

Listing 4: Grouping queries regarding their length

By storing the query hashes in a `std::unordered_map`, we can check the current `window_hash` against multiple queries due to `std::unordered_map.find()` with time complexity on average of $O(1)$ [?] (see ??).

```
for (auto const& [len, q_indices] : length_groups)

    std::unordered_map<uint64_t, std::vector<size_t>> hash_table;
    for (size_t q_idx : q_indices) {
        uint64_t h = 0;
        for (char c : queries[q_idx]) h = h * PRIME + (uint8_t)c;
        hash_table[h].push_back(q_idx);
    }

    //... preperation of data

    for (size_t j = 0; j <= text.length - len; ++j) {

        auto it = hash_table.find(window_hash);
        if (it != hash_table.end()) {
            for (size_t q_idx : it->second) {
                if (memcmp(&text[j], queries[q_idx].data(), len) == 0) {
                    indices[q_idx].push_back(j);
                }
            }
        }

        //... rolling hash implementation
    }
}
```

Listing 5: Check for the window-hash in the list of queries with the same length

3.4 Evaluation

4 Parallelization Across CPU Cores Using OpenMP

4.1 Candidate Selection and Validation

We use the two best bitmask-based approaches, `candidate_v3` and `candidate_v4`, in the implementations `candidate_openmp_v1` and `candidate_openmp_v2`, respectively. The queries are distributed across all CPU threads for both the selection and validation phases.

In the approach that creates a bitmask for each query, a separate bitmask is generated for every query, instead of creating a single bitmask once and reusing it for all queries. When using a single

large bitmask for all queries, the bitwise OR must be replaced with its atomic counterpart to avoid race conditions. OpenMP provides the directive `#pragma omp atomic` for this purpose. This data dependency introduces a disadvantage compared to the per-query bitmask approach.

For the validation phase, no synchronization is required, since each thread maintains its own vector within the overall result vector.

4.2 Rolling Hash

To parallelize the hash approach, we assign a group of queries of identical length to each thread. Since OpenMP requires a random access iterator or an integer index for the distribution of loops `#pragma omp parallel for`, a `std::unordered_map` cannot be used directly for parallelization. In the `hash_openmp_v1` implementation, we use a vector in which the query indexes are sorted by length. In addition, a second vector with a special block structure (`LengthBlock`) is created. This structure stores the respective word length, the start index within the sorted vector, and the number of words in this length group as metadata. This creates independent work packages that can be distributed efficiently by OpenMP to the available processor cores without any synchronization effort.

```
std::vector<size_t> q_indices(num_queries);
std::iota(q_indices.begin(), q_indices.end(), 0);

std::sort(q_indices.begin(), q_indices.end(), [&](size_t a, size_t b) {
    return queries[a].length() < queries[b].length();
});

std::vector<LengthBlock> blocks;
for (size_t i = 0; i < num_queries; ) {
    size_t len = queries[q_indices[i]].length();
    size_t start = i;
    while (i < num_queries && queries[q_indices[i]].length() == len) {
        i++;
    }
    if (len > 0 && len <= text_length) {
        blocks.push_back({len, start, i - start});
    }
}
```

Listing 6: New Data Struct

4.3 Reference Implementation

Finally, we also parallelized the reference implementation by distributing the queries across the threads. Using OpenMP, this modification required adding only a single line of code to the already very simple and concise reference implementation.

5 Using the GPU

5.1 OpenMPI

6 Summary